Career Intelligence MVP

Etap 3 - Backend, API Gateway, serwisy domenowe i kontrolery

Wersja operacyjna do implementacji w monorepo

Cel etapu: uruchomic warstwe backendowa dla czterech modulow MVP: Job Radar, Skill Signals, Applications Flow i Credits Visibility. Ten etap nie implementuje jeszcze pelnej AI/ML ani zewnetrznych platnosci, tylko tworzy stabilny szkielet API, kontrakty, kontrolery, warstwe domenowa i punkty integracji. Czyli robimy kregoslup, nie makiete z ladnym banerem i zerowa odpowiedzialnoscia.

1. Zakres Etapu 3

- API Gateway z prefiksem /v1 i przekierowaniem ruchu do serwisow domenowych.
- Wspolne typowane kontrakty API oparte o Zod.
- Auth middleware w Gateway: walidacja JWT i przekazanie userId do serwisow przez naglowek x-user-id.
- JobService: lista radarowa, szczegoly oferty, zapis oferty, analiza dopasowania.
- SkillService: sygnaly umiejetnosci, ekstrakcja skillow z tekstu, analiza skill gap dla oferty.
- ApplicationService: lista aplikacji, tworzenie aplikacji z oferty, zmiana statusu, event log, przypomnienia.
- CreditService: konto kredytowe, ledger, pakiety, rezerwacja, commit i anulowanie kosztu.
- Healthchecki i minimalne repozytoria DB dla kazdego serwisu.

2. Instalacja zaleznosci backendowych

```
pnpm add @fastify/http-proxy undici
pnpm add -D tsup

pnpm --filter @ci/api-gateway add @fastify/cors @fastify/helmet @fastify/jwt undici zod dotenv
pnpm --filter @ci/job-service add fastify zod dotenv @ci/db @ci/types
pnpm --filter @ci/skill-service add fastify zod dotenv @ci/db @ci/types
pnpm --filter @ci/application-service add fastify zod dotenv @ci/db @ci/types
pnpm --filter @ci/credit-service add fastify zod dotenv @ci/db @ci/types
pnpm --filter @ci/auth-service add fastify zod dotenv @fastify/jwt @ci/db @ci/types
```

3. Struktura folderow po Etapie 3

```
apps/api-gateway/src/
├─ index.ts
├─ env.ts
├─ plugins/
│  ├─ auth.ts
│  └─ error-handler.ts
├─ routes/
│  ├─ health.routes.ts
│  ├─ jobs.routes.ts
│  ├─ skills.routes.ts
│  ├─ applications.routes.ts
│  └─ credits.routes.ts
└─ proxy/
   └─ service-client.ts

services/job-service/src/
├─ index.ts
├─ routes/job.routes.ts
├─ domain/job-radar.service.ts
```

```
└─ repositories/job.repository.ts

services/skill-service/src/
├─ index.ts
├─ routes/skill.routes.ts
├─ domain/skill-signal.service.ts
└─ nlp/skill-extractor.ts

services/application-service/src/
├─ index.ts
├─ routes/application.routes.ts
├─ domain/application.service.ts
└─ repositories/application.repository.ts

services/credit-service/src/
├─ index.ts
├─ routes/credit.routes.ts
├─ domain/credit.service.ts
└─ repositories/credit.repository.ts

packages/types/src/
├─ api.ts
├─ jobs.ts
├─ skills.ts
├─ applications.ts
└─ credits.ts
```

4. Zmienne srodowiskowe

```
API_GATEWAY_PORT=4000
JOB_SERVICE_URL=http://localhost:4101
SKILL_SERVICE_URL=http://localhost:4102
APPLICATION_SERVICE_URL=http://localhost:4103
CREDIT_SERVICE_URL=http://localhost:4104
AUTH_SERVICE_URL=http://localhost:4105

JWT_SECRET=replace_me_in_real_environment
INTERNAL_SERVICE_TOKEN=local_internal_token_change_me
```

5. Wspolne typy API

packages/types/src/api.ts

```
export type ApiResponse<T> = {
  data: T;
  meta?: Record<string, unknown>;
};

export type ApiError = {
  code: string;
  message: string;
  details?: Record<string, unknown>;
};

export type PaginatedResponse<T> = ApiResponse<T[]> & {
  meta: {
    limit: number;
    offset: number;
    total?: number;
```

```
  };
};
```

packages/types/src/jobs.ts

```
export type JobRecommendedAction = "apply" | "save" | "reject" | "hide" | "learn_more";

export type JobRadarItem = {
  id: string;
  companyName: string;
  title: string;
  location: string | null;
  salaryMin: number | null;
  salaryMax: number | null;
  salaryCurrency: string | null;
  matchScore: number;
  valueScore: number;
  riskScore: number;
  recommendedAction: JobRecommendedAction;
  fitReasons: string[];
  riskReasons: string[];
  matchedSkills: string[];
  missingSkills: string[];
};

export type JobDetail = JobRadarItem & {
  description: string;
  sourceUrl: string | null;
  aiSummary: string | null;
};
```

packages/types/src/skills.ts

```
export type SkillSignalType = "strength" | "gap" | "market_trend" | "recommendation";

export type SkillSignal = {
  id: string;
  skillName: string | null;
  signalType: SkillSignalType;
  score: number;
  title: string;
  explanation: string;
  recommendation: string | null;
};

export type ExtractedSkill = {
  name: string;
  normalizedName: string;
  confidence: number;
};
```

packages/types/src/applications.ts

```
export type ApplicationStatus =
  | "draft"
  | "saved"
  | "applied"
  | "interview"
  | "offer"
  | "rejected"
  | "archived";

export type ApplicationItem = {
```

```
  id: string;
  jobId: string | null;
  companyName: string;
  roleTitle: string;
  status: ApplicationStatus;
  appliedAt: string | null;
  lastActivityAt: string | null;
  nextFollowUpAt: string | null;
  notes: string | null;
  updatedAt: string;
};

export type ApplicationEventItem = {
  id: string;
  applicationId: string;
  eventType: string;
  fromStatus: ApplicationStatus | null;
  toStatus: ApplicationStatus | null;
  message: string | null;
  createdAt: string;
};
```

packages/types/src/credits.ts

```
export type CreditAccountSummary = {
  balance: number;
  reservedBalance: number;
  availableBalance: number;
  lowBalanceThreshold: number;
  isLowBalance: boolean;
};

export type CreditLedgerItem = {
  id: string;
  direction: "debit" | "credit";
  category: string;
  status: "pending" | "posted" | "failed" | "cancelled";
  amount: number;
  description: string;
  sourceType: string | null;
  sourceId: string | null;
  occurredAt: string;
};

export type CreditPackageItem = {
  id: string;
  name: string;
  description: string | null;
  credits: number;
  priceCents: number;
  currency: string;
};
```

6. API Gateway

apps/api-gateway/src/env.ts

```
import dotenv from "dotenv";
import { z } from "zod";

dotenv.config();
```

```
const envSchema = z.object({
  API_GATEWAY_PORT: z.coerce.number().default(4000),
  JOB_SERVICE_URL: z.string().url(),
  SKILL_SERVICE_URL: z.string().url(),
  APPLICATION_SERVICE_URL: z.string().url(),
  CREDIT_SERVICE_URL: z.string().url(),
  AUTH_SERVICE_URL: z.string().url(),
  JWT_SECRET: z.string().min(16),
  INTERNAL_SERVICE_TOKEN: z.string().min(8),
});

export const env = envSchema.parse(process.env);
```

apps/api-gateway/src/plugins/auth.ts

```
import fp from "fastify-plugin";
import jwt from "@fastify/jwt";
import type { FastifyRequest } from "fastify";
import { env } from "../env";

declare module "fastify" {
  interface FastifyRequest {
    userId?: string;
  }
}

export const authPlugin = fp(async (app) => {
  await app.register(jwt, {
    secret: env.JWT_SECRET,
  });

  app.decorate("authenticate", async (request: FastifyRequest) => {
    const payload = await request.jwtVerify<{ sub: string }>();
    request.userId = payload.sub;
  });
});

declare module "fastify" {
  interface FastifyInstance {
    authenticate: (request: FastifyRequest) => Promise<void>;
  }
}
```

apps/api-gateway/src/proxy/service-client.ts

```
import { request } from "undici";
import { env } from "../env";

export async function callService<T>(input: {
  baseUrl: string;
  path: string;
  method?: "GET" | "POST" | "PATCH" | "DELETE";
  userId: string;
  body?: unknown;
  query?: Record<string, string | number | boolean | undefined>;
}): Promise<T> {
  const url = new URL(input.path, input.baseUrl);

  for (const [key, value] of Object.entries(input.query ?? {})) {
    if (value !== undefined) url.searchParams.set(key, String(value));
  }
```

```
  const response = await request(url, {
    method: input.method ?? "GET",
    headers: {
      "content-type": "application/json",
      "x-user-id": input.userId,
      "x-internal-service-token": env.INTERNAL_SERVICE_TOKEN,
    },
    body: input.body ? JSON.stringify(input.body) : undefined,
  });

  const text = await response.body.text();
  const json = text ? JSON.parse(text) : null;

  if (response.statusCode >= 400) {
    throw new Error(json?.message ?? `Service call failed with ${response.statusCode}`);
  }

  return json as T;
}
```

apps/api-gateway/src/routes/jobs.routes.ts

```
import type { FastifyInstance } from "fastify";
import { env } from "../env";
import { callService } from "../proxy/service-client";

export async function jobsRoutes(app: FastifyInstance) {
  app.get("/v1/jobs/radar", { preHandler: app.authenticate }, async (request) => {
    return callService({
      baseUrl: env.JOB_SERVICE_URL,
      path: "/internal/jobs/radar",
      userId: request.userId!,
      query: request.query as Record<string, string>,
    });
  });

  app.get("/v1/jobs/:id", { preHandler: app.authenticate }, async (request) => {
    const { id } = request.params as { id: string };
    return callService({
      baseUrl: env.JOB_SERVICE_URL,
      path: `/internal/jobs/${id}`,
      userId: request.userId!,
    });
  });

  app.post("/v1/jobs/:id/save", { preHandler: app.authenticate }, async (request) => {
    const { id } = request.params as { id: string };
    return callService({
      baseUrl: env.JOB_SERVICE_URL,
      path: `/internal/jobs/${id}/save`,
      method: "POST",
      userId: request.userId!,
      body: request.body,
    });
  });
}
```

apps/api-gateway/src/index.ts

```
import Fastify from "fastify";
import cors from "@fastify/cors";
```

```
import helmet from "@fastify/helmet";
import { env } from "./env";
import { authPlugin } from "./plugins/auth";
import { jobsRoutes } from "./routes/jobs.routes";
import { skillsRoutes } from "./routes/skills.routes";
import { applicationsRoutes } from "./routes/applications.routes";
import { creditsRoutes } from "./routes/credits.routes";

const app = Fastify({ logger: true });

await app.register(cors, { origin: true, credentials: true });
await app.register(helmet);
await app.register(authPlugin);

app.get("/health", async () => ({ status: "ok", service: "api-gateway" }));

await app.register(jobsRoutes);
await app.register(skillsRoutes);
await app.register(applicationsRoutes);
await app.register(creditsRoutes);

await app.listen({ port: env.API_GATEWAY_PORT, host: "0.0.0.0" });
```

7. Middleware wewnetrzny dla serwisow

```
// services/_shared/internal-auth.ts
import type { FastifyInstance, FastifyRequest } from "fastify";

export function registerInternalAuth(app: FastifyInstance) {
  app.addHook("preHandler", async (request: FastifyRequest, reply) => {
    if (request.url === "/health") return;

    const expected = process.env.INTERNAL_SERVICE_TOKEN;
    const actual = request.headers["x-internal-service-token"];
    const userId = request.headers["x-user-id"];

    if (!expected || actual !== expected) {
      return reply.code(401).send({ code: "UNAUTHORIZED", message: "Invalid service token." });
    }

    if (!userId || Array.isArray(userId)) {
      return reply.code(401).send({ code: "UNAUTHORIZED", message: "Missing user context." });
    }

    request.userId = userId;
  });
}
```

8. JobService

services/job-service/src/domain/job-radar.service.ts

```
import { z } from "zod";
import { JobRepository } from "../repositories/job.repository";

export const jobRadarQuerySchema = z.object({
  limit: z.coerce.number().int().min(1).max(100).default(20),
  offset: z.coerce.number().int().min(0).default(0),
```

```
  q: z.string().optional(),
  location: z.string().optional(),
  minMatchScore: z.coerce.number().int().min(0).max(100).optional(),
});

export class JobRadarService {
  constructor(private readonly repository = new JobRepository()) {}

  async getRadar(userId: string, query: unknown) {
    const parsed = jobRadarQuerySchema.parse(query);
    const items = await this.repository.findRadarItems(userId, parsed);

    return {
      data: items,
      meta: {
        limit: parsed.limit,
        offset: parsed.offset,
      },
    };
  }

  async getDetail(userId: string, jobId: string) {
    const detail = await this.repository.findJobDetail(userId, jobId);

    if (!detail) {
      throw new Error("Job not found.");
    }

    return { data: detail };
  }

  async saveJob(userId: string, jobId: string, note?: string) {
    const saved = await this.repository.saveJob(userId, jobId, note);
    return { data: saved };
  }
}
```

services/job-service/src/routes/job.routes.ts

```
import type { FastifyInstance } from "fastify";
import { JobRadarService } from "../domain/job-radar.service";

export async function jobRoutes(app: FastifyInstance) {
  const service = new JobRadarService();

  app.get("/internal/jobs/radar", async (request) => {
    return service.getRadar(request.userId!, request.query);
  });

  app.get("/internal/jobs/:id", async (request) => {
    const { id } = request.params as { id: string };
    return service.getDetail(request.userId!, id);
  });

  app.post("/internal/jobs/:id/save", async (request) => {
    const { id } = request.params as { id: string };
    const body = request.body as { note?: string } | undefined;
    return service.saveJob(request.userId!, id, body?.note);
  });
}
```

9. SkillService

```
// services/skill-service/src/nlp/skill-extractor.ts
const KNOWN_SKILLS = [
  "react",
  "typescript",
  "node.js",
  "nodejs",
  "postgresql",
  "mongodb",
  "redis",
  "python",
  "fastify",
];

export function extractSkillsFromText(text: string) {
  const lower = text.toLowerCase();

  return KNOWN_SKILLS
    .filter((skill) => lower.includes(skill))
    .map((skill) => ({
      name: skill === "nodejs" ? "Node.js" : skill,
      normalizedName: skill.replace(".", ""),
      confidence: 0.8,
    }));
}
// services/skill-service/src/routes/skill.routes.ts
import type { FastifyInstance } from "fastify";
import { z } from "zod";
import { extractSkillsFromText } from "../nlp/skill-extractor";

const extractInputSchema = z.object({ text: z.string().min(1) });

export async function skillRoutes(app: FastifyInstance) {
  app.get("/internal/skills/signals", async (request) => {
    // Repo implementation podlaczamy do skill_signals z Etapu 2.
    return { data: [], meta: { userId: request.userId } };
  });

  app.post("/internal/skills/extract", async (request) => {
    const input = extractInputSchema.parse(request.body);
    return { data: extractSkillsFromText(input.text) };
  });

  app.get("/internal/skills/jobs/:jobId/gap", async (request) => {
    const { jobId } = request.params as { jobId: string };
    return {
      data: {
        jobId,
        strengths: [],
        gaps: [],
        recommendations: [],
      },
    };
  });
}
```

10. ApplicationService

```
// services/application-service/src/domain/application.service.ts
import { z } from "zod";
```

```
export const createApplicationInputSchema = z.object({
  jobId: z.string().uuid().optional(),
  companyName: z.string().min(1),
  roleTitle: z.string().min(1),
  source: z.string().optional(),
  notes: z.string().optional(),
});

export const updateStatusInputSchema = z.object({
  status: z.enum(["draft", "saved", "applied", "interview", "offer", "rejected", "archived"]),
  message: z.string().optional(),
});

export class ApplicationService {
  async list(userId: string) {
    return { data: [], meta: { userId } };
  }

  async create(userId: string, body: unknown) {
    const input = createApplicationInputSchema.parse(body);
    return { data: { id: crypto.randomUUID(), userId, ...input, status: "saved" } };
  }

  async updateStatus(userId: string, applicationId: string, body: unknown) {
    const input = updateStatusInputSchema.parse(body);
    return {
      data: {
        id: applicationId,
        userId,
        status: input.status,
        event: input.message ?? `Status changed to ${input.status}`,
      },
    };
  }
}
// services/application-service/src/routes/application.routes.ts
import type { FastifyInstance } from "fastify";
import { ApplicationService } from "../domain/application.service";

export async function applicationRoutes(app: FastifyInstance) {
  const service = new ApplicationService();

  app.get("/internal/applications", async (request) => {
    return service.list(request.userId!);
  });

  app.post("/internal/applications", async (request) => {
    return service.create(request.userId!, request.body);
  });

  app.patch("/internal/applications/:id/status", async (request) => {
    const { id } = request.params as { id: string };
    return service.updateStatus(request.userId!, id, request.body);
  });
}
```

11. CreditService

```
// services/credit-service/src/domain/credit.service.ts
```

```
import { z } from "zod";

const reserveInputSchema = z.object({
  amount: z.number().int().positive(),
  reason: z.string().min(1),
  sourceType: z.string().optional(),
  sourceId: z.string().optional(),
});

export class CreditService {
  async getAccount(userId: string) {
    return {
      data: {
        userId,
        balance: 0,
        reservedBalance: 0,
        availableBalance: 0,
        lowBalanceThreshold: 10,
        isLowBalance: true,
      },
    };
  }

  async getLedger(userId: string) {
    return { data: [], meta: { userId } };
  }

  async getPackages() {
    return { data: [] };
  }

  async reserve(userId: string, body: unknown) {
    const input = reserveInputSchema.parse(body);
    return { data: { id: crypto.randomUUID(), userId, ...input, status: "pending" } };
  }
}
// services/credit-service/src/routes/credit.routes.ts
import type { FastifyInstance } from "fastify";
import { CreditService } from "../domain/credit.service";

export async function creditRoutes(app: FastifyInstance) {
  const service = new CreditService();

  app.get("/internal/credits/account", async (request) => service.getAccount(request.userId!));
  app.get("/internal/credits/ledger", async (request) => service.getLedger(request.userId!));
  app.get("/internal/credits/packages", async () => service.getPackages());
  app.post("/internal/credits/reservations", async (request) => service.reserve(request.userId!,
request.body));
}
```

12. Route maps w API Gateway

```
Publiczne endpointy Gateway:

GET    /health

GET    /v1/jobs/radar
GET    /v1/jobs/:id
POST   /v1/jobs/:id/save
```

```
GET    /v1/skills/signals
POST   /v1/skills/extract
GET    /v1/skills/jobs/:jobId/gap

GET    /v1/applications
POST   /v1/applications
PATCH  /v1/applications/:id/status

GET    /v1/credits/account
GET    /v1/credits/ledger
GET    /v1/credits/packages
POST   /v1/credits/reservations
```

13. Uruchamianie backendu

```
pnpm db:up

pnpm --filter @ci/api-gateway dev
pnpm --filter @ci/job-service dev
pnpm --filter @ci/skill-service dev
pnpm --filter @ci/application-service dev
pnpm --filter @ci/credit-service dev
```

14. Testy smoke przez curl

```
curl http://localhost:4000/health
curl http://localhost:4101/health
curl http://localhost:4102/health
curl http://localhost:4103/health
curl http://localhost:4104/health

# Przyklad z JWT, token musi miec sub=userId
curl -H "Authorization: Bearer $JWT" http://localhost:4000/v1/jobs/radar
curl -H "Authorization: Bearer $JWT" http://localhost:4000/v1/credits/account
```

15. Kryteria ukonczenia Etapu 3

- API Gateway startuje i odpowiada na /health.
- Kazdy serwis domenowy startuje i odpowiada na /health.
- Gateway wymaga JWT dla endpointow /v1/* i przekazuje userId do serwisow.
- Serwisy akceptuja tylko ruch z poprawnym INTERNAL_SERVICE_TOKEN.
- Istnieja endpointy dla Job Radar, Skill Signals, Applications Flow i Credits Visibility.
- Kontrakty typow sa wyciagniete do packages/types.
- Kazdy endpoint zwraca przewidywalny ksztalt { data, meta? } albo blad { code, message }.
- Kod jest gotowy do podpiecia repozytoriow Drizzle/Mongo/Redis z Etapu 2.

16. Co zostaje na Etap 4

- Frontend routing, layout aplikacji i provider API.
- Client SDK albo fetch wrapper dla Gateway.
- Zarzadzanie stanem po stronie web.
- Pierwsze prawdziwe ekrany pod Job Radar, Skill Signals, Applications Flow i Credits Visibility.
- Podpiecie danych z endpointow Etapu 3 do komponentow UI.